Programación para la administración de sistemas en C

Manual del alumnado de UF1286 · IFCT0609 · Edición revisada de noviembre de 2026

Este manual utiliza C como lenguaje principal para aprender a construir herramientas sencillas de administración de sistemas. Parte del razonamiento en pseudocódigo y avanza hacia variables, decisiones, bucles, funciones, archivos y servicios del sistema operativo. Los ejemplos se acompañan de resultados y comprobaciones para entender qué hace cada programa.

Qué cambia en esta edición

Los ejemplos principales se han reescrito en C. Se amplían las explicaciones sobre int main(void), caracteres, cadenas, parámetros y retorno para dar continuidad a las clases. Python queda reservado a una comparación final de consulta; no es necesario aprender su sintaxis para seguir el manual.

Alcance del material

Se conservan los ejes del manual anterior: estructura y servicios del sistema operativo, lenguajes y bibliotecas, requisitos, ciclo de vida, calidad, memoria, procesos y optimización. El caso Aula Norte conecta la programación con necesidades de un centro formativo. Es material didáctico orientado a UF1286; la correspondencia es orientativa y no acredita por sí sola todos los criterios oficiales.

Los fundamentos utilizan C11. Los capítulos de carpetas, espacio libre y procesos se identifican como Linux POSIX, porque esas interfaces no forman parte de C estándar. Las prácticas se realizan con archivos ficticios y una cuenta normal.

Cómo estudiar cada ejemplo

Describe las entradas y salidas. Predice el resultado. Compila y ejecuta. Compara lo obtenido con lo esperado y cambia una sola cosa. Guarda el mensaje completo si aparece un error. Los programas completos están también en el paquete de ejemplos; las actividades propuestas no incluyen solucionario.

Para practicar Punto de reanudación de clase: capítulos 13 a 15, después de comprobar variables, decisiones y bucles. El resto del recorrido permanece disponible como consulta y desarrollo posterior.

Ruta de aprendizaje

Organización del manual y uso del paquete de ejemplos

Bloque	Capítulos y propósito
Comprender el entorno	1 a 4 · Problemas, procesos, servicios y sistema operativo.
Construir programas	5 a 15 · Compilación, datos, operadores, bucles y funciones.
Organizar datos y código	16 a 21 · Punteros, estructuras, módulos, archivos y formatos.
Administrar y comprobar	22 a 28 · Errores, recursos, procesos, memoria y requisitos.
Integrar y mantener	29 a 36 · Proyecto, pruebas, calidad, entrega y siguientes unidades.
Consolidar	37 a 40 · Prácticas, revisión, glosario y comparación con Python.

Qué contiene el paquete

Cada archivo .c identificado en el manual es un programa independiente, salvo los tres archivos del ejemplo modular. Los archivos inventario.c, 20_pruebas.c y LEEME.md completan el proyecto y su verificación. Los ejemplos evitan datos reales del alumnado.

Convenciones

Un bloque titulado Pseudocódigo explica una solución; no se compila. Un bloque con #include y main suele ser un programa completo. Una orden de terminal tampoco se pega dentro del archivo .c. Los textos de los ejemplos usan caracteres sencillos para facilitar su ejecución en consolas con distintas configuraciones.

En Linux las órdenes de ejecución empiezan por ./ y en PowerShell suelen empezar por .\. El compilador, las bibliotecas y las cabeceras deben estar disponibles en el entorno acordado con el profesor. Las extensiones del editor no sustituyen al compilador.

Para practicar Crea una carpeta de trabajo propia y separa código, datos ficticios e informes. Comprueba que puedes explicar para qué sirve cada archivo antes de añadir otro.

1 Del problema al programa

Programar empieza por precisar qué necesita una persona.

Aula Norte quiere recibir un aviso cuando un disco tenga menos del 15 % libre. Antes de programar hay que decidir qué unidad se consulta, cómo se obtiene su capacidad y qué ocurre si la medida es inválida. Una frase general no es todavía un requisito comprobable.

Un algoritmo describe pasos finitos y ordenados. El pseudocódigo expresa esos pasos sin depender de la sintaxis de un lenguaje. Un programa los representa en un lenguaje ejecutable, como C. El mismo razonamiento puede implementarse en varios lenguajes.

Elemento	Ejemplo
Entradas	Capacidad 200 y espacio libre 20, en la misma unidad.
Proceso	Validar los datos y calcular libre / capacidad × 100.
Salida	10 % libre y estado AVISO.
Frontera	Con 30 libres de 200 resulta 15 % y no hay aviso.

Pseudocódigo

```
LEER capacidad, libre
SI capacidad <= 0 O libre < 0 O libre > capacidad ENTONCES
    MOSTRAR "Datos invalidos"
SINO
    porcentaje <- libre / capacidad * 100
    SI porcentaje < 15 ENTONCES
        MOSTRAR "AVISO"
    SINO
        MOSTRAR "OK"
    FIN SI
FIN SI
```

La validación impide dividir entre cero o presentar un porcentaje imposible. El umbral usa menor que, no menor o igual. Una traza en papel permite detectar esa diferencia antes de escribir C.

Para practicar Define las entradas y salidas de un contador de archivos. Aclara si cuenta subcarpetas, enlaces y elementos que no puede consultar. No confundas contar nombres con poder leer el contenido.

2 Programa proceso y servicio

Distingue lo que está guardado, lo que se ejecuta y lo que se ofrece.

Un programa es código preparado para realizar una tarea. Su archivo puede estar en el disco sin estar ejecutándose. Cuando se inicia, el sistema operativo crea un proceso con memoria, identificador y recursos asociados. Varias ejecuciones del mismo programa pueden originar procesos distintos.

Un servicio es una capacidad disponible para usuarios u otras aplicaciones. Puede depender de varios procesos. El servicio de matriculación requiere que la aplicación, la base de datos y el almacenamiento colaboren. Ver un proceso activo no demuestra que una matrícula pueda guardarse.

Concepto	Ejemplo en el aula
Programa	El ejecutable inventario compilado desde inventario.c.
Proceso	Una ejecución de inventario sobre datos_demo.
Servicio	Consulta de plazas o acceso a documentos compartidos.
Recurso	Tiempo de CPU, memoria, disco, conexión o permiso.

Cómo colaboran los recursos

La CPU ejecuta instrucciones. La RAM mantiene datos y código en uso. El almacenamiento conserva archivos. El sistema operativo coordina el acceso y aplica permisos. La red interviene cuando los datos viajan a otro equipo. Un síntoma puede tener varias causas, por lo que conviene comprobar cada hipótesis con una evidencia.

Si un informe se guarda localmente y falla al guardarlo en una carpeta compartida, la comparación orienta hacia el acceso remoto, sus permisos o su almacenamiento. Todavía no demuestra qué elemento concreto falla. Si falla también en local, hay que revisar la aplicación y el entorno.

Para practicar Propón una prueba que compruebe si un servidor responde y otra que compruebe si un documento se guarda y puede recuperarse. Explica qué demuestra cada prueba.

3 El sistema operativo como intermediario

Los programas solicitan servicios mediante interfaces definidas.

El núcleo del sistema operativo organiza mecanismos esenciales: planificación de procesos, protección de memoria y acceso a dispositivos. Las aplicaciones ordinarias no deben controlar sin restricciones toda la memoria o cualquier dispositivo. La separación de privilegios reduce el alcance de muchos errores.

Capa	Responsabilidad
Tu programa	Decide qué archivo necesita y cómo tratar sus datos.
Biblioteca	Ofrece funciones reutilizables como fopen o printf.
Interfaz del sistema	Permite solicitar servicios del núcleo de forma controlada.
Dispositivo y controlador	Realizan el acceso específico al soporte físico.

Una interfaz de programación, o API, define operaciones y reglas. Una biblioteca reúne implementaciones reutilizables. Una llamada al sistema permite entrar de manera controlada en un servicio del núcleo. No son sinónimos: una función que suma dos números puede resolverse sin una llamada al núcleo.

Qué ocurre al abrir un archivo

La aplicación solicita abrir una ruta. El sistema comprueba que puede resolverla y que los permisos permiten la operación. Si la apertura funciona, la biblioteca entrega un recurso para trabajar con el archivo. Si falla, el programa debe comunicarlo y decidir si puede continuar.

En C, fopen devuelve un puntero a FILE o NULL si no puede abrir. FILE es una abstracción de la biblioteca; no es el contenido completo del archivo ni un controlador de disco. Al terminar se llama a fclose y se comprueba su resultado si hay escrituras pendientes.

Para practicar Dibuja el recorrido de guardar un informe desde tu programa hasta el almacenamiento. Indica dónde puede aparecer un error de ruta y dónde un problema de permisos.

4 Estructura y servicios del sistema operativo

Las arquitecturas distribuyen responsabilidades de distintas maneras.

Organización	Qué conviene comprender
Monolítica	Muchas funciones del núcleo comparten un espacio privilegiado. Un fallo allí puede tener un impacto amplio.
Modular	Permite incorporar partes mediante interfaces definidas. Sigue habiendo dependencias.
Micronúcleo	Reduce las funciones del núcleo y sitúa otros servicios en procesos separados. La comunicación cobra importancia.
Por capas	Organiza servicios sobre interfaces sucesivas para separar responsabilidades.

Estas descripciones no son categorías totalmente excluyentes. Un sistema real puede combinar varias decisiones. Lo importante para una herramienta administrativa es saber qué servicio necesita, cómo lo solicita y cómo reconoce un fallo.

Servicios que utilizará nuestro programa

La gestión de archivos permite abrir, leer, escribir y consultar metadatos. La gestión de procesos permite iniciar y esperar tareas. La memoria ofrece almacenamiento durante la ejecución. Los mecanismos de comunicación conectan procesos. Los permisos y la identidad limitan las operaciones autorizadas.

La prioridad de un proceso no garantiza por sí sola que termine antes: influyen esperas de disco, bloqueos y carga. Tampoco más memoria asignada implica mejor diseño. Las medidas deben interpretarse junto a la tarea concreta y a los recursos de los que depende.

Para practicar Para un inventario que solo consulta nombres y tamaños, enumera los servicios necesarios. Explica por qué no necesita permisos de administrador ni instalar un controlador.

5 Preparar compilar y ejecutar

C necesita un compilador y una fase de construcción del ejecutable.

Guarda el código en un archivo con extensión .c. El preprocesador atiende directivas como #include; el compilador traduce el código y el enlazador une las piezas necesarias para formar el ejecutable. La compilación puede señalar errores y advertencias, pero no demuestra que el algoritmo sea correcto.

01_hola.c

```
#include <stdio.h>

int main(void) {
    printf("Hola, aula\n");
    return 0;
}
```

Órdenes en Linux con GCC

```
gcc -std=c11 -Wall -Wextra -Wpedantic 01_hola.c -o hola
./hola
```

En Windows con GCC configurado

```
gcc -std=c11 -Wall -Wextra -Wpedantic 01_hola.c -o hola.exe
.\hola.exe
```

La segunda orden de Windows corresponde a PowerShell. Los ejemplos Linux POSIX posteriores necesitan un entorno compatible, como Linux o WSL; no basta con cambiar la extensión del ejecutable. Code Runner o C/C++ Compile Run facilitan órdenes desde el editor, pero requieren las herramientas configuradas.

Para practicar Compila y comprueba la salida Hola, aula. Después provoca un error quitando un punto y coma, lee el diagnóstico y restaura el programa. No continúes con una versión antigua del ejecutable si la compilación nueva ha fallado.

6 Variables tipos y asignación

En C se declara el tipo del objeto que guarda un dato.

Una variable con nombre permite conservar un valor durante la ejecución. Declarar int cantidad reserva un objeto adecuado para un entero; asignar cantidad = 4 cambia su valor. Conviene inicializar las variables antes de leerlas. Un valor sin inicializar no es una fuente fiable de datos.

Tipo	Uso inicial	Formato en printf
int	Enteros dentro del rango de int	%d
double	Números con parte decimal	%f o %.2f
char	Un carácter como A	%c
char[]	Texto terminado en \0	%s
size_t	Tamaños e índices de objetos	%zu

02_variables.c

```
#include <stdio.h>
int main(void) {
    int cantidad = 4;
    double precio = 25.0;
    char grupo = 'A';
    char nombre[] = "Ana";
    double total = precio * cantidad;
    printf("%s del grupo %c\n", nombre, grupo);
    printf("Total: %.2f\n", total);
    return 0;
}
```

El resultado es Ana del grupo A y Total: 100.00. Las comillas simples delimitan una constante de carácter; las dobles, una cadena. Los rangos de los tipos dependen de la implementación; no presupongas que int siempre tiene exactamente 32 bits.

Para practicar Traza a = 5; b = a; a = 8. ¿Qué valor conserva b? Añade printf para comprobarlo. Después explica qué variable del ejemplo representa una cantidad y cuál un importe.

7 Operadores y expresiones

Los operadores calculan, comparan y combinan condiciones.

Pseudocódigo	C	Ejemplo
Asignar <-	=	total = precio * cantidad;
Igual / distinto	== / !=	cantidad == 4
Y / O / NO	&& / || / !	edad >= 18 && edad <= 65
Resto	%	17 % 5 produce 2
División	/	9 / 2 produce 4; 9.0 / 2 produce 4.5

Los paréntesis hacen visible el orden que necesitas. 7 + 3 * 2 produce 13 y (7 + 3) * 2 produce 20. En división entre enteros el resultado se trunca hacia cero. Convertir el resultado después no recupera la parte decimal perdida.

03_operadores.c

```c
#include <stdio.h>
int main(void) {
    int a = 9, b = 2;
    double resultado = (double)a / b;
    printf("Entera: %d\n", a / b);
    printf("Real: %.1f\n", resultado);
    printf("Resto: %d\n", a % b);
    return 0;
}
```

La conversión (double)a hace que la división se realice con aritmética de coma flotante. El operador ^ de C es una operación exclusiva bit a bit; no representa una potencia. Para elevar al cuadrado un dato sencillo, escribe x * x.

Comprueba el divisor antes de dividir o calcular un resto. Los enteros tienen límites: sumar o multiplicar valores grandes puede desbordar. En estos primeros ejemplos se usan valores pequeños y acotados; una utilidad real debe controlar esos rangos.

Para practicar Predice y comprueba las tres líneas del programa: 4, 4.5 y 1. Explica por qué = y == no se pueden intercambiar.

8 Leer entradas y validar conversiones

La entrada de teclado es texto que debe convertirse y comprobarse.

scanf puede convertir una entrada sencilla. Su retorno indica cuántas conversiones se completaron. Con %lf espera la dirección de un double; con %d, la de un int. El símbolo & obtiene la dirección donde la función escribirá el dato. Con printf se pasa el valor; con esta lectura se pasa su dirección.

04_entrada.c

```
#include <stdio.h>
int main(void) {
    int cantidad;
    printf("Cantidad de equipos entre 1 y 100: ");
    if (scanf("%d", &cantidad) != 1) {
        fprintf(stderr, "Entrada no numerica\n");
        return 1;
    }
    if (cantidad < 1 || cantidad > 100) {
        fprintf(stderr, "Cantidad fuera de rango\n");
        return 1;
    }
    printf("Equipos: %d\n", cantidad);
    return 0;
}
```

Este programa sirve para practicar una lectura de un entero representable. No es un analizador estricto: puede aceptar 12abc como 12 y no ofrece un control adecuado de entradas numéricas desmesuradas. Si falla, finaliza; no repite sin retirar el texto pendiente.

Progresión de validación

Primero comprueba que se convirtió el dato. Después verifica las reglas del problema. Más adelante, para entradas completas y control de rango, usa fgets con strtol y revisa el final de la cadena y errno. El capítulo 22 presenta esa alternativa.

Para practicar Prueba 4, 0 y hola. Anota la salida y el código de finalización. Explica por qué validar la conversión no equivale a validar que la cantidad sea aceptable.

9 Decisiones y condiciones compuestas

Una condición selecciona qué instrucciones se ejecutan.

Pseudocódigo

```
SI porcentaje < 5 ENTONCES
    MOSTRAR "CRITICO"
SINO SI porcentaje < 15 ENTONCES
    MOSTRAR "AVISO"
SINO
    MOSTRAR "OK"
FIN SI
```

05_decisiones.c

```c
#include <stdio.h>
int main(void) {
    double porcentaje = 10.0;
    if (porcentaje < 5.0) {
        puts("CRITICO");
    } else if (porcentaje < 15.0) {
        puts("AVISO");
    } else {
        puts("OK");
    }
    return 0;
}
```

La salida es AVISO. El orden importa: si primero se comprobara menor que 15, un 4 entraría ahí y no llegaría al caso crítico. Las llaves agrupan las instrucciones de cada rama; se mantienen incluso si solo hay una instrucción.

Para comprobar un intervalo usa edad >= 18 && edad <= 65. La forma matemática 18 <= edad <= 65 no expresa ese intervalo en C: la primera comparación produce 0 o 1, y después se compara ese resultado con 65.

Para practicar Comprueba 4, 5, 14, 15 y 16. Justifica qué rama se ejecuta en los límites. Añade una validación previa para impedir porcentajes negativos o mayores que 100.

10 Repetir con while

MIENTRAS comprueba la condición antes de ejecutar el cuerpo.

Un contador permite repetir un número de veces. Necesita un valor inicial, una condición de continuación y una actualización que acerque a la salida. Si la condición empieza siendo falsa, el cuerpo no se ejecuta.

Pseudocódigo

```
contador <- 1
MIENTRAS contador <= 5 HACER
    MOSTRAR contador
    contador <- contador + 1
FIN MIENTRAS
```

06_while.c

```c
#include <stdio.h>
int main(void) {
    int contador = 1;
    while (contador <= 5) {
        printf("%d\n", contador);
        contador = contador + 1;
    }
    return 0;
}
```

Momento	Valor y decisión
Primera comprobación	1 <= 5: se muestra 1 y se actualiza a 2.
Última vuelta	5 <= 5: se muestra 5 y se actualiza a 6.
Salida	6 <= 5 es falso. No se muestra 6.

contador++ incrementa el contador en uno y puede sustituir la asignación de este ejemplo. Un punto y coma inmediatamente detrás de while puede crear un cuerpo vacío; revisa esa línea si el programa parece detenido.

Para practicar Cambia el inicio a 6 y predice el resultado. Después escribe un bucle que muestre del 5 al 1; señala qué actualización evita que sea infinito.

11 Validar con do while

REPETIR HASTA QUE se traduce invirtiendo la condición.

do while ejecuta primero el cuerpo y comprueba después si debe repetir. En pseudocódigo HASTA QUE se detiene cuando su condición es verdadera; en C while continúa mientras su condición sea verdadera. Por eso las condiciones no se copian literalmente.

Pseudocódigo

```
REPETIR
    LEER nota
HASTA QUE nota >= 0 Y nota <= 10
```

07_do_while.c

```c
#include <stdio.h>
int main(void) {
    int nota;
    do {
        printf("Nota entera de 0 a 10: ");
        if (scanf("%d", &nota) != 1) {
            fprintf(stderr, "Entrada no numerica\n");
            return 1;
        }
    } while (nota < 0 || nota > 10);
    printf("Nota aceptada: %d\n", nota);
    return 0;
}
```

Con las entradas -1 y 7 se solicita la nota dos veces y se acepta 7. Una entrada no numérica termina esta versión: así no queda atrapada releyendo el mismo texto. Las limitaciones de scanf son las explicadas en el capítulo 8.

El punto y coma final de do while es obligatorio. Es distinto del punto y coma accidental tras la condición de un while ordinario. El cuerpo usa llaves para que se vea qué se repite.

Para practicar Escribe la condición de continuación para aceptar una cantidad entre 1 y 20. Comprueba 0, 1, 20 y 21. Explica por qué se necesita O en la condición que rechaza los extremos.

12 Recorrer con for y acumular

PARA concentra inicio, condición y actualización.

for resulta útil cuando sabes cómo avanza un índice. En C los tres apartados se separan con punto y coma. El acumulador conserva un resultado parcial; debe inicializarse antes del bucle. Si se reinicia dentro de cada vuelta, se pierde lo anterior.

08_for.c

```
#include <stdio.h>
int main(void) {
    int suma = 0;
    for (int i = 1; i <= 5; i++) {
        suma += i;
        printf("i=%d suma=%d\n", i, suma);
    }
    printf("Total: %d\n", suma);
    return 0;
}
```

i	suma después de la vuelta
1	1
2	3
3	6
4	10
5	15

suma += i equivale aquí a suma = suma + i. El índice i se declara en el for y deja de estar disponible al terminar ese bucle. Un contador de elementos y una suma de sus valores son datos distintos, aunque ambos cambien durante el mismo recorrido.

Del pseudocódigo a C

PARA i DESDE 1 HASTA 5 con ambos extremos incluidos se expresa como for (int i = 1; i <= 5; i++). Para recorrer cinco posiciones de un array, normalmente se usan índices de 0 a 4 y la condición i < 5.

Para practicar Muestra la tabla del 7 del 1 al 10. Después calcula la suma de los números pares del 2 al 20. Predice los resultados antes de ejecutar.

13 Entender int main y el retorno

El tipo de retorno no limita los tipos que puedes utilizar dentro.

09_main.c

```
#include <stdio.h>
int main(void) {
    char nombre[] = "Bea";
    char grupo = 'A';
    double nota = 8.5;
    printf("%s, grupo %c, nota %.1f\n", nombre, grupo, nota);
    return 0;
}
```

Parte	Significado
int	main devuelve un valor entero al entorno de ejecución.
main	Nombre de la función de entrada de este programa.
void entre paréntesis	La función no recibe parámetros en esta forma.
Llaves	Delimitan el cuerpo de la función.
return 0	Termina main comunicando finalización correcta.

La salida visible es Bea, grupo A, nota 8.5. El valor devuelto por main es 0, no ese texto. printf comunica información por la salida estándar; return entrega un resultado a quien llamó a la función. Son acciones diferentes.

Dentro de una función que devuelve int puedes declarar char, double, arrays y otros objetos. El tipo escrito antes del nombre describe el resultado de la función, no todos los datos internos. return "Bea" no corresponde a un retorno int.

Si necesitas recibir argumentos

La forma habitual es int main(int argc, char *argv[]). argc cuenta los argumentos, incluido el nombre del programa; argv permite acceder a sus textos. No se eligen dos parámetros numéricos arbitrarios para main: una función propia, como sumar, sí puede definirlos.

Para practicar Explica por separado qué se muestra en pantalla y qué se devuelve. Cambia solo nombre y nota: ¿hay alguna razón para cambiar el int de main?

14 Crear funciones con parámetros

Una función permite dar nombre a una tarea y reutilizarla.

Pseudocódigo

```
FUNCION sumar(a, b) DEVUELVE ENTERO
    DEVOLVER a + b
FIN FUNCION
resultado <- sumar(3, 5)
MOSTRAR resultado
```

10_funciones.c

```c
#include <stdio.h>
int sumar(int a, int b) {
    return a + b;
}
int main(void) {
    int resultado = sumar(3, 5);
    printf("Resultado: %d\n", resultado);
    return 0;
}
```

a y b son parámetros: nombres locales que reciben datos al llamar a la función. 3 y 5 son argumentos: valores enviados en esa llamada. La función calcula 8 y lo devuelve; main lo almacena en resultado y lo muestra. El ejemplo usa enteros pequeños para evitar desbordamientos.

C pasa los argumentos por valor. En esta llamada, modificar a no cambiaría una variable del llamador. Para modificar un objeto externo se pasa una dirección y se accede a él mediante un puntero, como se explica en el capítulo 17.

Contrato y alcance

El contrato describe entradas válidas, salida y posibles errores. Las variables locales de sumar no existen como tales fuera de la función. Si defines una función después de main, declara antes su prototipo, por ejemplo int sumar(int a, int b);.

Para practicar Crea double calcular_total(double precio, int cantidad). Usa datos válidos y pequeños. Haz dos llamadas con valores distintos y explica por qué se reutiliza la misma función.

15 Devolver valores y mostrar mensajes

Conviene separar el cálculo de la presentación.

11_porcentaje.c

```
#include <stdio.h>
double porcentaje_libre(double total, double libre) {
    return libre / total * 100.0;
}
void mostrar_estado(double porcentaje) {
    if (porcentaje < 15.0) {
        puts("AVISO");
    } else {
        puts("OK");
    }
}
int main(void) {
    double total = 200.0, libre = 20.0;
    if (total <= 0 || libre < 0 || libre > total) {
        fputs("Medida invalida\n", stderr);
        return 1;
    }
    double porcentaje = porcentaje_libre(total, libre);
    printf("Libre: %.1f %%\n", porcentaje);
    mostrar_estado(porcentaje);
    return 0;
}
```

porcentaje_libre devuelve un double y no imprime. mostrar_estado devuelve void: no entrega un valor, aunque sí produce una salida visible. main valida los datos antes de llamar al cálculo. El contrato de porcentaje_libre exige valores finitos válidos y total positivo.

La salida es Libre: 10.0 % y AVISO. %% permite mostrar el símbolo de porcentaje en un formato de printf. Si cambias libre a 30, el resultado será 15.0 % y OK.

Para practicar Explica dónde se realiza cada responsabilidad: validar, calcular, clasificar y mostrar. Añade una función de clasificación que devuelva un código, para que la presentación pueda decidir qué texto mostrar.

16 Arrays cadenas y estructuras

Agrupa datos relacionados y respeta siempre sus límites.

Un array contiene un número determinado de elementos del mismo tipo. int notas[4] tiene posiciones 0, 1, 2 y 3. C no comprueba automáticamente que el índice sea válido. Acceder a notas[4] está fuera de sus límites y el comportamiento deja de estar definido por el lenguaje.

12_arrays.c

```
#include <stdio.h>
int main(void) {
    int notas[] = {4, 5, 7, 8};
    int suma = 0, aprobados = 0;
    for (int i = 0; i < 4; i++) {
        suma += notas[i];
        if (notas[i] >= 5) aprobados++;
    }
    printf("Media: %.1f\n", suma / 4.0);
    printf("Aprobados: %d\n", aprobados);
    return 0;
}
```

La media es 6.0 y hay 3 aprobados. El número de elementos no se deduce automáticamente dentro de una función que recibe un parámetro como int notas[]; normalmente también se pasa la cantidad de elementos.

Cadenas y registros

char nombre[4] = "Ana" almacena A, n, a y el terminador \0. Una cadena necesita espacio para ese byte final. Para comparar contenido se utiliza strcmp, declarado en string.h, no ==. Las funciones de texto requieren cadenas válidas y límites adecuados.

```
/* Fragmento de declaracion, no programa independiente. */
struct Equipo {
    char codigo[16];
    int ram_gib;
};
struct Equipo aula = {"PC01", 8};
```

Para practicar Representa cinco equipos ficticios con código y RAM. Cuenta los que tienen menos de 8 GiB y decide cómo representar una cantidad desconocida sin confundirla con cero.

17 Punteros y duración de los objetos

Una dirección debe apuntar a un objeto válido del tipo adecuado.

Un puntero guarda una dirección. & obtiene la dirección de un objeto y * permite acceder al objeto apuntado. Son operadores con un papel distinto de la multiplicación, según el contexto. Tener una dirección no garantiza que sea válida ni que el objeto siga existiendo.

13_punteros.c

```
#include <stdio.h>
void incrementar(int *valor) {
    *valor = *valor + 1;
}
int main(void) {
    int cantidad = 3;
    int *direccion = &cantidad;
    incrementar(direccion);
    printf("Cantidad: %d\n", cantidad);
    return 0;
}
```

Expresión	Interpretación
cantidad	Objeto entero que empieza valiendo 3.
&cantidad	Dirección de ese objeto.
direccion	Puntero que conserva esa dirección.
***direccion**	Acceso al entero; tras la llamada vale 4.

La función recibe una copia del puntero, pero ambas direcciones se refieren al mismo entero. El contrato de incrementar exige un puntero válido a un int modificable; no acepta NULL. Esta condición debe quedar documentada o comprobarse en una interfaz que admita ese caso.

No devuelvas la dirección de una variable local automática: deja de existir al terminar la función. No accedas a memoria liberada ni fuera de un array. Una dirección impresa solo tiene sentido en esa ejecución; no la copies para reutilizarla en otro programa.

Para practicar Dibuja cantidad y direccion como dos celdas distintas. Explica por qué scanf recibe &cantidad y printf recibe cantidad cuando se utiliza %d.

18 Módulos cabeceras y bibliotecas

Separa el contrato público de la implementación.

Un archivo de cabecera .h declara lo que otros archivos necesitan conocer. Un archivo .c implementa las funciones. #include incorpora declaraciones durante el preprocesado; el enlazador necesita además la implementación correspondiente. No se resuelve una función propia simplemente incluyendo stdio.h.

medidas.h

medidas.h

```
#ifndef MEDIDAS_H
#define MEDIDAS_H
double porcentaje_libre(double total, double libre);
#endif
```

medidas.c

medidas.c

```
#include "medidas.h"
double porcentaje_libre(double total, double libre) {
    return libre / total * 100.0;
}
```

modular.c

modular.c

```
#include <stdio.h>
#include "medidas.h"
int main(void) {
    printf("%.1f\n", porcentaje_libre(200, 30));
    return 0;
}

gcc -std=c11 -Wall -Wextra -Wpedantic modular.c medidas.c -o modular
```

La salida es 15.0. La protección con #ifndef evita repetir las declaraciones al incluir la cabecera varias veces. La función conserva el contrato del capítulo 15. stdio.h ofrece entrada y salida; stdlib.h, conversiones y memoria; string.h, operaciones con cadenas; time.h, tiempo.

Para practicar Decide dónde colocarías una función de cálculo, la lectura de argumentos y la escritura de un informe. Evita declarar variables globales para comunicar datos que pueden viajar mediante parámetros.

19 Rutas carpetas y permisos

La ruta relativa parte del directorio de trabajo del proceso.

Una ruta absoluta parte de una raíz. Una relativa se interpreta desde el directorio de trabajo, que no tiene por qué coincidir con la carpeta del código ni con la del ejecutable. Ejecutar el mismo programa desde dos carpetas puede producir resultados distintos si abre datos.txt con una ruta relativa.

Entorno	Ejemplo de ruta	Observación
Linux	/home/alumno/practicas/datos.txt	Ruta absoluta desde /.
Windows	C:\Practicas\datos.txt	Ruta absoluta de una unidad.
Relativa	datos_demo/a.txt	Depende de dónde se inició el programa.

En una cadena de C, la barra inversa introduce secuencias especiales: "C:\\Practicas\\datos.txt" representa barras inversas literales. En argumentos escritos en la terminal, las reglas de comillas dependen del intérprete de órdenes. Pon entre comillas una ruta que contenga espacios.

Consultar no equivale a leer contenido

Un directorio puede permitir listar nombres y no leer ciertos archivos. Consultar tamaño y tipo necesita permisos distintos de abrir el contenido. Un error puede significar inexistencia, denegación, ruta demasiado larga o fallo de acceso; no conviertas todos esos casos en «no existe».

C estándar ofrece fopen para archivos, pero no una API general para enumerar directorios. En este manual se utiliza dirent.h en los ejemplos Linux POSIX. En Windows nativo se necesita una implementación específica, por ejemplo las interfaces de búsqueda de archivos de Win32.

Para practicar Crea una carpeta de laboratorio con dos archivos y una subcarpeta. Describe su estructura y prueba cómo cambia una ruta relativa al ejecutar desde el directorio padre. No cambies permisos de carpetas reales del centro.

20 Leer y escribir archivos de texto

Comprueba la apertura, las operaciones y el cierre.

14_escribir.c

```
#include <stdio.h>
int main(void) {
    FILE *f = fopen("nota_demo.txt", "wx");
    if (f == NULL) {
        perror("Crear nota_demo.txt");
        return 1;
    }
    int fallo = fprintf(f, "Revision de laboratorio\n") < 0;
    if (fclose(f) == EOF) fallo = 1;
    if (fallo) {
        fputs("No se completo la escritura\n", stderr);
        return 1;
    }
    puts("Archivo creado");
    return 0;
}
```

El modo wx, disponible en C11, intenta crear un archivo nuevo y falla si ya existe. El modo w puede truncar uno existente; a añade al final y r abre para lectura. No elijas w si el requisito exige conservar un informe anterior. Un fallo de escritura puede aparecer al cerrar porque quedan datos pendientes.

Lectura incremental

```
/* Fragmento: f ya esta abierto con "r". */
char linea[256];
while (fgets(linea, sizeof linea, f) != NULL) {
    fputs(linea, stdout);
}
/* Comprobar ferror(f) y cerrar f al terminar. */
```

fgets limita los bytes que guarda y añade el terminador si lee con éxito. Si una línea supera el espacio disponible, se lee por fragmentos: no confundas automáticamente cada llamada con una línea completa. La codificación y los saltos de línea también forman parte del formato.

Para practicar Ejecuta dos veces el programa de creación. La segunda debe fallar sin borrar la nota. Para repetir con éxito, conserva primero una copia o usa otro nombre ficticio.

21 Datos estructurados con CSV y JSON

Un formato acordado permite intercambiar información.

CSV representa registros con campos. Hay variantes de separador, codificación y salto de línea; para el proyecto se elige coma y campos de texto entre comillas. Si un nombre contiene comillas dobles, cada una se escribe dos veces. Un simple printf("%s,%d") no resuelve todos los nombres posibles.

```
/* Ejemplo de contenido CSV, no codigo C. */
"nombre","bytes"
"a.txt",4
"informe,noviembre.txt",12
"nota ""final"".txt",8
```

JSON como formato alternativo

```
{"codigo":"PC01","ram_gib":8,"verificado":true}
```

JSON diferencia cadenas, números, booleanos, arrays y objetos. C estándar no incluye un analizador JSON. Una aplicación real puede utilizar una biblioteca apropiada y revisar sus reglas de memoria, errores y licencia. Construir JSON concatenando texto sin escapar no es una alternativa general válida.

Decisión	Contrato del proyecto
Campos	nombre y bytes; cada fila es un archivo regular.
Texto	Nombres tratados como bytes; para abrirlos como UTF-8 se exige un conjunto de nombres UTF-8 válidos.
Número	Tamaño lógico decimal no negativo.
Orden	El recorrido didáctico no promete orden alfabético.

Un archivo CSV puede contener campos que una hoja de cálculo interprete como fórmulas. Entrecomillar no neutraliza esa interpretación. En el laboratorio usamos nombres ficticios sencillos; al importar datos externos se debe elegir un tratamiento explícito de texto.

Para practicar Escribe a mano el CSV de un archivo llamado ventas,enero.txt. Explica por qué separar la línea simplemente por comas pierde la estructura.

22 Errores y conversión estricta

C utiliza valores de retorno y convenciones de error.

C no dispone de try y except como mecanismo general de su biblioteca estándar. Cada función define cómo comunica fallos: fopen devuelve NULL, una lectura informa EOF y muchas interfaces del sistema usan -1. errno solo se interpreta cuando una función documentada ha indicado un error; una llamada exitosa no tiene por qué ponerlo a cero.

15_conversion.c

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <ctype.h>
int main(void) {
    char texto[64], *fin;
    if (fgets(texto, sizeof texto, stdin) == NULL) return 1;
    /* Rechazar lineas que no caben completas. */
    int i = 0;
    while (texto[i] && texto[i] != '\n') i++;
    if (texto[i] != '\n' && !feof(stdin)) {
        fputs("Linea demasiado larga\n", stderr); return 1;
    }
    errno = 0;
    long n = strtol(texto, &fin, 10);
    if (fin == texto || errno == ERANGE) return 1;
    while (isspace((unsigned char)*fin)) fin++;
    if (*fin != '\0' || n < 1 || n > 100) return 1;
    printf("Cantidad: %ld\n", n);
    return 0;
}
```

strtol deja en fin la posición donde terminó. Se rechaza la ausencia de dígitos, el desbordamiento, el texto sobrante y el valor fuera del intervalo. Se permiten espacios finales. Esta versión puede rechazar de forma conservadora una entrada larga sin salto final aunque no sobrepasara el número de caracteres útiles.

Para practicar Prueba 12, 12abc, una línea vacía, 0, 100 y un número enorme. Diferencia los errores de formato de los errores de rango. Añade mensajes específicos sin cambiar qué se acepta.

23 Consultar espacio de almacenamiento

Linux POSIX utiliza statvfs para consultar un sistema de archivos.

16_disco_posix.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <sys/statvfs.h>
int main(void) {
    struct statvfs datos;
    if (statvfs(".", &datos) == -1) {
        perror("statvfs");
        return 1;
    }
    if (datos.f_blocks == 0) return 1;
    double total = (double)datos.f_blocks * datos.f_frsize;
    double disponible = (double)datos.f_bavail * datos.f_frsize;
    double gib = 1024.0 * 1024.0 * 1024.0;
    double porcentaje = disponible / total * 100.0;
    printf("Total: %.2f GiB\n", total / gib);
    printf("Disponible: %.2f GiB\n", disponible / gib);
    printf("Porcentaje disponible: %.1f %%\n", porcentaje);
    return 0;
}
```

La ruta punto consulta el sistema de archivos del directorio actual. No enumera todos los discos físicos. f_bavail representa bloques disponibles para usuarios sin privilegios; puede diferir de f_bfree por reservas. Por eso el dato se llama disponible y no simplemente libre.

El cálculo convierte a double antes de multiplicar para no multiplicar primero dos enteros potencialmente grandes. Es una aproximación para presentar medidas, no un contador entero exacto de bytes enormes. Cuotas, permisos y cambios simultáneos pueden limitar lo que realmente se puede escribir.

De una muestra a monitorización

Una ejecución describe un instante. Monitorizar requiere acordar periodicidad, guardar muestras, definir umbrales y decidir la respuesta. Una consulta excesivamente frecuente también consume recursos. No afirmes que el disco está sano por obtener un porcentaje aceptable.

Para practicar Separa consulta, cálculo y presentación. Documenta qué unidad se consulta y qué significa disponible. Prueba primero el cálculo con 100 y 14, 15 y 16 antes de utilizar datos reales.

24 Lanzar procesos y esperar resultados

Ejemplo específico de Linux POSIX con un proceso hijo.

fork crea un proceso hijo y devuelve valores distintos en cada proceso. exec sustituye la imagen del proceso por otro programa. waitpid permite esperar al hijo e interpretar su finalización. Un hijo puede arrancar y terminar con error; también puede tardar más de lo previsto.

17_proceso_posix.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <errno.h>
int main(void) {
    pid_t hijo = fork();
    if (hijo == -1) { perror("fork"); return 1; }
    if (hijo == 0) {
        execlp("printf", "printf", "%s\n", "5", (char *)NULL);
        perror("exec");
        _exit(127);
    }
    int estado;
    while (waitpid(hijo, &estado, 0) == -1) {
        if (errno != EINTR) { perror("waitpid"); return 1; }
    }
    if (!WIFEXITED(estado)) return 1;
    printf("Codigo del hijo: %d\n", WEXITSTATUS(estado));
    return WEXITSTATUS(estado) == 0 ? 0 : 1;
}
```

En un entorno con la utilidad printf disponible, el hijo escribe 5 y el padre muestra Codigo del hijo: 0. Los argumentos se pasan separados; no hay intérprete de órdenes evaluando una cadena. execlp busca el ejecutable en PATH: el ejemplo presupone un entorno de laboratorio de confianza.

Esta versión no captura la salida ni fija tiempo máximo: el hijo hereda la consola y el padre espera. Son requisitos adicionales que necesitarían tuberías y control de espera. No uses system con una orden construida a partir de texto introducido por usuarios.

Para practicar Distingue fallo al crear el hijo, fallo al sustituirlo y código no cero del programa ejecutado. Explica qué habría que añadir para limitar el tiempo de espera.

25 Registros y trazabilidad

Un registro documenta eventos; un informe resume resultados.

Un mensaje útil identifica momento, nivel, operación y contexto suficiente. INFO puede registrar un inicio; AVISO, una omisión; ERROR, una operación fallida. Son convenciones del proyecto, no palabras que el sistema interprete automáticamente. No incluyas contraseñas ni información personal innecesaria.

18_registro.c

```
#include <stdio.h>
#include <time.h>
int main(void) {
    time_t ahora = time(NULL);
    if (ahora == (time_t)-1) return 1;
    struct tm *local = localtime(&ahora);
    char fecha[32];
    if (local == NULL) return 1;
    if (strftime(fecha, sizeof fecha,
                 "%Y-%m-%d %H:%M:%S", local) == 0) return 1;
    FILE *f = fopen("actividad_demo.log", "a");
    if (f == NULL) { perror("registro"); return 1; }
    int fallo = fprintf(f, "%s INFO Inicio de revision\n", fecha) < 0;
    if (fclose(f) == EOF) fallo = 1;
    return fallo ? 1 : 0;
}
```

El archivo se abre para añadir y la fecha depende del equipo. Cada ejecución añade una línea. Una herramienta de producción necesita una política sobre crecimiento, conservación, acceso y rotación. Si varias tareas escriben a la vez, no presupongas que sus mensajes largos quedarán indivisibles.

El informe podría decir «12 archivos y 1 omisión». El registro podría explicar qué consulta falló, en qué ejecución y por qué. Una ruta completa puede revelar datos personales: utiliza identificadores mínimos si bastan para diagnosticar. Registrar que se llamó a una función no demuestra que su efecto se completara.

Para practicar Redacta mensajes de inicio, consulta omitida y fin parcial. Incluye un identificador ficticio de ejecución y explica qué datos excluirías.

26 Memoria y procesamiento por partes

La eficiencia depende de qué conservas y durante cuánto tiempo.

Las variables automáticas suelen durar hasta salir de su bloque. La memoria dinámica obtenida con malloc permanece hasta que se libera o termina el proceso. Un fallo de reserva devuelve NULL. sizeof permite expresar el tamaño del tipo sin asumir cuántos bytes ocupa en cada plataforma.

19_memoria.c

```
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    size_t cantidad = 4;
    int *notas = malloc(cantidad * sizeof *notas);
    if (notas == NULL) return 1;
    for (size_t i = 0; i < cantidad; i++) notas[i] = 5;
    printf("Primera nota: %d\n", notas[0]);
    free(notas);
    notas = NULL;
    return 0;
}
```

Aquí cantidad es un valor pequeño fijado en el programa. Si procede de una entrada, hay que limitarlo y comprobar que el producto no desborde size_t antes de reservar. No liberes dos veces el mismo bloque ni sigas utilizándolo después de free.

Leer por partes

Para contar avisos de un registro grande se puede leer una porción, actualizar un contador y descartarla. La memoria deja de crecer con todo el archivo. Si el formato exige líneas completas, hay que detectar las que no caben en el búfer; buscar una palabra por fragmentos puede perder coincidencias divididas entre dos lecturas.

La memoria virtual relaciona espacios de direcciones con memoria física y otros mecanismos. No convierte el almacenamiento en RAM igual de rápida. Si hay intercambio intenso de páginas, el rendimiento puede empeorar. Mide antes de atribuirlo al tamaño de memoria del equipo.

Para practicar Compara guardar todos los registros en un array con procesarlos de uno en uno. Indica qué opción permite ordenar después y qué coste de memoria introduce.

27 Concurrencia y recursos compartidos

El orden de las operaciones puede cambiar el resultado.

Dos tareas concurrentes progresan en intervalos solapados; no necesariamente se ejecutan al mismo instante. Los hilos de un proceso suelen compartir memoria. Los procesos tienen espacios de direcciones distintos, aunque pueden compartir archivos y otros recursos mediante mecanismos concretos.

Paso	Tarea A	Tarea B
1	Lee contador 5	
2		Lee contador 5
3	Calcula y escribe 6	
4		Calcula y escribe 6

Se esperaba 7, pero queda 6. El incremento combina lectura, cálculo y escritura; no se vuelve indivisible por escribir contador++. En C, accesos concurrentes no sincronizados a un objeto compartido, con al menos una escritura, pueden constituir una carrera de datos y producir comportamiento indefinido.

Coordinar exige un diseño

Un bloqueo puede proteger una sección crítica. Una cola puede repartir tareas. Los bloqueos mal ordenados pueden provocar un interbloqueo, en el que cada tarea espera un recurso retenido por otra. Añadir hilos tampoco garantiza una mejora si el cuello de botella es el almacenamiento.

El proyecto del manual es secuencial para poder explicar su comportamiento. Dos ejecuciones podrían intentar crear el mismo informe: una comprobación de existencia seguida de apertura con w no protege de esa carrera. El modo de creación exclusiva debe formar parte de la operación de apertura.

Para practicar Propón una política de nombres para no mezclar informes de dos grupos. Explica por qué una copia de archivos no se vuelve atómica solo por ejecutarse en otro hilo.

28 Requisitos diseño y ciclo de vida

Un criterio verificable orienta la implementación y las pruebas.

Un requisito funcional describe una capacidad. Uno de calidad establece condiciones como no modificar originales, comunicar omisiones o cumplir un tiempo bajo una carga definida. «Que funcione bien» no permite decidir si una prueba ha superado el objetivo.

Identificador	Contrato del proyecto en C
R1	Recibir carpeta e informe como argumentos.
R2	Consultar archivos regulares inmediatos y omitir subcarpetas y enlaces.
R3	Escribir nombre y tamaño lógico en bytes, sin prometer orden.
R4	Crear un CSV nuevo sin sobrescribir otro; situarlo fuera del origen.
R5	No alterar originales y advertir de consultas omitidas.
R6	Retornar 0 si completa, 1 ante fallo y 2 si hay omisiones.

Esta versión cambia de forma explícita un requisito del proyecto anterior: escribe durante el recorrido y no ordena los nombres. Reduce memoria y complejidad inicial. Si se necesita salida alfabética, habrá que almacenar y ordenar registros o adoptar otra técnica; no basta con asumir que readdir ya los devuelve ordenados.

Ciclo de vida

Analizar, diseñar, implementar, probar, desplegar y mantener forman un ciclo con revisiones. Una prueba puede revelar un requisito incompleto y obligar a volver al diseño. Conserva versiones pequeñas con mensajes que expliquen la intención y una forma de recuperar la última conocida.

La calidad incluye corrección, robustez, mantenibilidad, portabilidad, seguridad y eficiencia. Una herramienta rápida que omite archivos sin avisar no cumple su contrato. Un programa que compila sin advertencias todavía necesita pruebas.

Para practicar Añade un requisito de filtro por extensión. Define qué ocurre con .TXT y .txt y con un filtro sin coincidencias. Redacta tres casos de aceptación.

29 Preparar el proyecto de inventario

El proyecto completo está en inventario.c dentro del paquete.

La utilidad consulta una carpeta y escribe un CSV nuevo fuera de ella. Se ejecuta en Linux POSIX. Solo incluye archivos regulares inmediatos: no entra en subcarpetas ni sigue enlaces simbólicos. Consulta metadatos, no abre el contenido de cada archivo. Un archivo cuyo contenido no pueda leerse podría contarse si sus metadatos sí pueden consultarse.

Datos ficticios reproducibles

```
mkdir datos_demo
printf hola > datos_demo/a.txt
printf adios > datos_demo/b.txt
mkdir datos_demo/subcarpeta
```

Estas órdenes de terminal crean archivos de 4 y 5 bytes, sin salto de línea. Utiliza una carpeta de laboratorio nueva: la redirección con > sustituye un archivo existente. La subcarpeta debe quedar fuera del CSV. El total esperado es 2 archivos y 9 bytes.

Decisión	Efecto
Consulta con fstatat	Obtiene tipo y tamaño en relación con el directorio abierto.
AT_SYMLINK_NOFOLLOW	Consulta el enlace como enlace, sin seguirlo.
Apertura con wx	Evita sobrescribir un archivo ya existente.
Escritura durante el recorrido	No conserva una lista de todos los nombres ni los ordena.

El código comprueba si el padre resuelto del informe coincide con el origen. Al ser un recorrido inmediato, un informe en una subcarpeta no se incorpora al resultado. Aun así, en el aula se guardará en la carpeta de trabajo exterior para mantener separados datos e informes.

El inventario no es una fotografía atómica: una carpeta activa puede cambiar durante la consulta. La comprobación de rutas no es una defensa completa frente a cambios deliberados concurrentes de directorios o montajes. El proyecto se limita al laboratorio controlado.

Para practicar Dibuja la carpeta de trabajo, datos_demo y el CSV exterior. Predice qué debe pasar si se intenta guardar el informe directamente dentro de datos_demo.

30 Entender el recorrido de archivos

Estos fragmentos explican el código completo; no se compilan por separado.

```
errno = 0;
struct dirent *entrada = readdir(dir);
if (entrada == NULL) {
    if (errno != 0) { perror("readdir"); return -1; }
    break;
}
```

dir es el directorio abierto con opendir. readdir entrega una entrada o NULL. NULL puede indicar fin normal o error; poner errno a cero justo antes permite diferenciarlos. Las entradas . y .. se descartan porque representan el directorio actual y su padre.

```
struct stat datos;
if (fstatat(dirfd(dir), entrada->d_name, &datos,
            AT_SYMLINK_NOFOLLOW) == -1) {
    perror("Consulta omitida");
    (*omitidos)++;
    continue;
}
if (!S_ISREG(datos.st_mode)) continue;
```

dirfd obtiene el descriptor asociado al directorio abierto. fstatat consulta la entrada respecto a él, sin construir una ruta larga concatenando cadenas. El tipo de archivo se comprueba con S_ISREG. No se confía en d_type porque no está disponible de la misma forma en todos los sistemas ni siempre tiene un tipo conocido.

Un error al consultar una entrada incrementa omitidos y permite seguir. Un fallo al leer el directorio completo es fatal porque impide saber qué queda. El archivo completo añade comprobaciones de desbordamiento a los contadores: el fragmento anterior se simplifica para explicar la decisión.

Contar o recorrer recursivamente

Esta versión no implementa recursión. Para contar subcarpetas y su contenido habría que definir una política de enlaces, profundidad, errores y recursos abiertos. No añadas una llamada recursiva sin decidir cómo se evita seguir ciclos o salir del alcance de la práctica.

Para practicar Explica las diferencias entre omitir por alcance una subcarpeta, omitir por error una consulta y terminar por un fallo del recorrido.

31 Entender la escritura y los códigos de salida

El informe solo se anuncia como completado si se verifica el cierre.

```
FILE *salida = fopen(argv[2], "wx");
if (salida == NULL) {
    perror("Informe");
    closedir(dir);
    return 1;
}
```

El informe se crea con exclusividad. Comprobar antes si existe y abrir después con w sería una secuencia vulnerable a carreras. Aquí se solicita la condición en la apertura. El proyecto conserva un informe parcial si falla después; no anuncia éxito y no borra automáticamente un resultado que puede ayudar a investigar.

```
/* Parte final del programa; las variables ya existen. */
int fallo = inventariar(dir, salida, &archivos,
                        &bytes, &omitidos) == -1;
if (closedir(dir) == -1) fallo = 1;
if (fclose(salida) == EOF) fallo = 1;
if (fallo) return 1;
return omitidos ? 2 : 0;
```

Código	Interpretación
0	Recorrido e informe completados sin errores de consulta.
1	Argumentos inválidos o fallo; el informe puede no existir o ser parcial.
2	Informe completado, pero hubo consultas omitidas por error.

El operador condicional condición ? valor_si_verdadero : valor_si_falso permite elegir 2 o 0 según haya omisiones. No confunde un resultado parcial con uno completo. Los nombres y tamaños se escriben con escape CSV; los enteros grandes usan uintmax_t y PRIuMAX.

Un orden distinto de filas no es un error en este contrato. Enlaces y carpetas se omiten por alcance y no incrementan el contador de errores. El tamaño es lógico: no equivale al espacio ocupado en bloques ni al total del disco.

Para practicar Explica qué código devolverías si no existe la carpeta, si el CSV ya existe y si solo falla una consulta de metadatos pero el resto del informe se escribe correctamente.

32 Ejecutar y comprobar el proyecto

Trabaja desde la carpeta que contiene el código y datos_demo.

```
gcc -std=c11 -Wall -Wextra -Wpedantic inventario.c -o inventario
./inventario datos_demo informe_demo.csv
echo $?
```

echo $? muestra en una terminal de tipo Bash el código de la orden inmediatamente anterior. Con los datos preparados debe mostrar 0. Si una ruta contiene espacios, encierra cada argumento entre comillas.

Resultado esperado

```
Archivos: 2; bytes: 9
Consultas omitidas: 0
```

Caso	Comprobación
Dos archivos de prueba	Dos registros, tamaños 4 y 5, total 9.
Carpeta vacía	CSV con cabecera y cero registros.
Origen inexistente	Mensaje y código 1.
Origen que es archivo	Rechazo al abrir como directorio.
Informe existente	Código 1 y contenido anterior conservado.
Informe en origen	Rechazo antes de crear el CSV.
Enlace simbólico	No se incluye ni se sigue.
Nombre con coma o comillas	Conservación del nombre al analizar el CSV.

El paquete incluye verificar_proyecto.py como herramienta auxiliar de comprobación para el docente. No es un ejemplo que deba estudiar el alumnado: crea datos temporales y contrasta el ejecutable C. La enseñanza y la implementación del inventario siguen siendo C.

Para practicar Añade c.txt con 3 bytes, utiliza un informe con nombre nuevo y predice el total 12. Comprueba los datos de origen antes y después y documenta cualquier diferencia.

33 Pruebas y depuración

Compara comportamiento con expectativas antes de ampliar.

Las pruebas unitarias revisan funciones pequeñas; las de integración comprueban colaboraciones, como lectura y escritura; las de aceptación contrastan los requisitos del usuario. Una prueba debe observar resultados y efectos, no solo afirmar que se ejecutó una función.

20_pruebas.c

```
#include <assert.h>
#include <math.h>
#include <stdio.h>
double porcentaje(double total, double libre) {
    return libre / total * 100.0;
}
int main(void) {
    assert(fabs(porcentaje(200, 20) - 10.0) < 0.000001);
    assert(fabs(porcentaje(200, 30) - 15.0) < 0.000001);
    puts("Comprobaciones superadas");
    return 0;
}
```

assert sirve aquí para verificar expectativas del ejemplo, no para validar entradas de una herramienta. Puede desactivarse al compilar con NDEBUG. Se usa una tolerancia porque las operaciones de coma flotante pueden introducir redondeos. El contrato sigue exigiendo total positivo y datos válidos.

Depurar con evidencias

Si esperas 9 bytes y obtienes 0, revisa la carpeta consultada, las entradas seleccionadas y los retornos. Si falta un archivo, comprueba su tipo y los errores. Reduce el caso hasta poder trazarlo. Un depurador permite detener la ejecución y observar variables sin llenar el programa de mensajes permanentes.

```
gcc -std=c11 -g -Wall -Wextra -Wpedantic 11_porcentaje.c -o porcentaje
```

La opción -g incorpora información de depuración. Herramientas como GDB y los analizadores de memoria pueden ayudar según el entorno. Un fallo de prueba señala una diferencia; no identifica automáticamente la línea responsable.

Para practicar Diseña pruebas para cero, uno y varios registros, un nombre con coma y un destino ya existente. Anota resultado esperado, observado y conclusión.

34 Optimizar con medidas

Una mejora debe conservar el comportamiento acordado.

Primero identifica el coste que importa: tiempo transcurrido, CPU, memoria o acceso a disco. Mide una versión de referencia con una carga definida. Cambia una sola decisión y compara varias ejecuciones, dejando constancia del equipo y de otras tareas que puedan influir.

```
/* Fragmento Linux POSIX: rodea el trabajo que se mide. */
struct timespec inicio, fin;
if (clock_gettime(CLOCK_MONOTONIC, &inicio) == -1) return 1;
/* Aqui se ejecuta la tarea. */
if (clock_gettime(CLOCK_MONOTONIC, &fin) == -1) return 1;
double segundos = (double)(fin.tv_sec - inicio.tv_sec)
    + (fin.tv_nsec - inicio.tv_nsec) / 1000000000.0;
printf("Duracion: %.6f segundos\n", segundos);
```

El fragmento requiere time.h y habilitar las interfaces POSIX. CLOCK_MONOTONIC sirve para medir intervalos sin depender de los ajustes ordinarios del reloj de calendario. clock() de C mide tiempo de CPU de otra forma; no intercambies los dos conceptos en un informe.

Memoria y ordenación

El proyecto recorre n entradas y escribe los registros seleccionados. No retiene todos sus nombres. Si añadimos un orden alfabético en memoria, necesitaremos conservarlos y ordenar: un algoritmo habitual añade un coste del orden de n log n. Estas estimaciones describen crecimiento, no segundos universales.

Una caché guarda resultados para reutilizarlos, pero puede devolver datos antiguos. Si conservas durante horas el espacio disponible, un aviso puede llegar tarde. Define caducidad, invalidación y qué hacer si la nueva consulta falla.

Para practicar Propón una mejora del inventario y explica qué coste reduce, qué prueba garantiza equivalencia y en qué escenario no aportaría ventaja. No uses solo dos archivos para justificar rendimiento en producción.

35 Documentar entregar y mantener

Otra persona debe poder ejecutar tu versión con datos ficticios.

Campo	Ficha del proyecto
Nombre y versión	Inventario de carpeta en C edición didáctica 1.0.
Finalidad	Listar nombres y tamaños de archivos regulares inmediatos.
Requisitos	Linux POSIX, compilador C11 y permisos de consulta y creación.
Uso	inventario CARPETA INFORME.csv
Salida	CSV nombre y bytes, resumen y código 0, 1 o 2.
Límites	Sin recursión, sin orden garantizado y sin instantánea atómica.
Datos de prueba	a.txt de 4 bytes y b.txt de 5, con informe exterior nuevo.

Incluye código, instrucciones, casos de prueba y decisiones que afectan al usuario. Un comentario explica el motivo de una decisión o un contrato, no necesita traducir cada punto y coma. La documentación debe corresponder a la versión entregada.

Preparar una actualización

Conserva la versión anterior y su configuración, prueba en laboratorio y acuerda cómo verificar el resultado. Si afecta a datos, prepara una copia recuperable y una ventana de intervención. Volver al ejecutable anterior no deshace automáticamente cambios de datos.

Git conserva cambios e historia. GitHub aloja repositorios remotos; no es el compilador ni ejecuta de forma general un programa C dentro de una página estática. Publicar fuentes en el aula permite descargarlas, pero el alumnado debe compilarlas en su entorno.

Para practicar Redacta una guía de uso de media página. Una pareja distinta debe poder ejecutar tu versión sin preguntarte qué rutas, compilador o argumentos necesita.

36 Componentes y siguientes unidades

Una interfaz clara permite reutilizar una capacidad.

C organiza programas mediante funciones, estructuras y archivos. No incorpora clases como parte del lenguaje. Una estructura agrupa datos; funciones que trabajan con ella pueden separar responsabilidades. Esa organización no convierte automáticamente el código en un componente listo para integrar.

```
/* Fragmento de una interfaz, no programa completo. */
struct MedidaDisco {
    double total;
    double disponible;
};
int validar_medida(struct MedidaDisco medida);
double calcular_porcentaje(struct MedidaDisco medida);
```

La interfaz debe documentar unidades, valores aceptados, resultado y errores. Un consumidor no debería necesitar conocer los detalles internos para utilizarla. Si modificas el contrato, tendrás que revisar a quienes dependen de él. Una versión y un conjunto de pruebas ayudan a mantener esa compatibilidad.

Relación con el certificado

UF1287 profundiza en dispositivos y controladores; UF1288, en servicios de comunicaciones. MF0965_3 desarrolla diseño, implementación, integración y despliegue por componentes. Este capítulo introduce vocabulario y criterios; no sustituye esas unidades.

Para exponer un inventario a otra aplicación, elige un contrato útil para ella: una función declarada en una cabecera, un CSV con esquema o una interfaz de comunicación. Un texto libre pensado para una persona puede ser difícil de analizar de forma estable.

Para practicar Define la interfaz de una función que clasifique espacio disponible. Indica unidades, validación, retorno y dos pruebas. Después explica qué documentación necesitaría otra aplicación para utilizarla.

37 Prácticas progresivas para entregar

Entrega código C, datos ficticios, pruebas y una explicación breve.

Práctica 1 Clasificar medidas

Recibe capacidad y espacio disponible, valida y calcula el porcentaje. Clasifica CRITICO por debajo de 5 %, AVISO desde 5 % hasta menos de 15 % y OK a partir de 15 %. Separa cálculo y presentación. Aporta pruebas de frontera y entradas inválidas. Para entrada estricta usa la técnica del capítulo 22.

Práctica 2 Inventario lógico del aula

Representa cinco equipos ficticios con estructura, código y RAM en GiB. Identifica los que tienen menos de 8 GiB y los no verificados. Exporta a CSV. Comprueba códigos duplicados y explica cómo diferencias un dato desconocido de una cantidad cero.

Práctica 3 Analizar un registro

Diseña un formato sencillo con fecha, nivel y mensaje. Cuenta eventos por nivel y decide cómo tratar líneas incompletas o demasiado largas. No clasifiques una línea solo porque aparece ERROR en su mensaje: comprueba el campo de nivel. Incluye una entrada inválida.

Práctica 4 Ampliar el inventario

Añade un filtro de extensión y un tamaño mínimo en bytes. Define el tratamiento de mayúsculas y minúsculas. Conserva la creación sin sobrescritura y el código parcial. Un filtro sin coincidencias debe producir una cabecera válida con cero filas.

Práctica 5 Contar subcarpetas

Diseña primero una versión recursiva para Linux POSIX. Define si cuenta la carpeta inicial, cómo trata enlaces, qué profundidad máxima admite y qué hace ante un error. Implementa por etapas, sin seguir enlaces simbólicos. Presenta una estructura de carpetas con un resultado conocido.

Revisión entre parejas

Intercambia una versión con otra pareja. Ejecutad los casos acordados y anotad dos mejoras justificadas. Conservad los originales y no modifiquéis el código ajeno sin acordarlo. El informe debe distinguir lo observado de las hipótesis.

Para practicar Ampliación para quien avance más rápido: estudia cómo añadir orden alfabético o menor consumo de memoria manteniendo el contrato elegido. Aporta medidas y pruebas, no solo más opciones.

38 Revisión y autoevaluación

Explicar el comportamiento es parte de resolver la práctica.

Pregunta	Evidencia
¿Entiendo la necesidad?	Entradas, salidas, límites y criterio de aceptación.
¿El cálculo es correcto?	Traza con valores pequeños y casos de frontera.
¿Gestiono errores?	Ejecución inválida con mensaje y código.
¿Respeto datos originales?	Comparación de contenido antes y después.
¿Controlo recursos?	Cierres y liberaciones en éxito y en fallo.
¿Se puede reutilizar?	Funciones con contrato y llamadas explicadas.
¿Otra persona puede ejecutarlo?	Guía, versión y datos ficticios.
¿La mejora está demostrada?	Medidas comparables y resultados equivalentes.

Errores que debes reconocer

Confundir = y ==; escribir 18 <= edad <= 65; olvidar actualizar un contador; reiniciar la suma dentro del bucle; dividir enteros esperando decimales; leer una variable sin inicializar; olvidar & en scanf; usar %f en lugar de %lf para leer un double; salir de los límites de un array; devolver la dirección de una variable local; no comprobar fopen o fclose; confundir falta de permisos con inexistencia.

Una explicación final de un minuto

Cuenta qué problema resuelve tu herramienta, qué recibe, qué produce y qué no hace. Muestra una prueba con resultado conocido. Señala una limitación real y una mejora razonada. Explica qué datos se muestran y qué valor devuelve main.

Para practicar Si necesitas refuerzo, vuelve a variables, decisiones y trazas. Si avanzas con soltura, mejora validación, pruebas y documentación antes de aumentar funcionalidades. Las soluciones de estas actividades se trabajan con el docente por separado.

39 Glosario de consulta

Términos para entender el código y explicar decisiones.

Término	Significado
Algoritmo	Secuencia finita y ordenada que resuelve una tarea.
Argumento	Valor que se proporciona en una llamada.
Array	Conjunto de elementos del mismo tipo con posiciones indexadas.
Biblioteca	Implementación reutilizable de funciones y otros recursos.
Cabecera	Archivo con declaraciones para compartir una interfaz.
Cadena	Secuencia de caracteres terminada en el byte nulo.
Compilación	Traducción que participa en la construcción del ejecutable.
Contrato	Condiciones de entrada, resultado, errores y alcance.
Descriptor	Identificador de un recurso abierto en interfaces POSIX.
Enlazado	Unión de unidades compiladas y bibliotecas.
Metadato	Dato sobre un archivo, como tamaño o tipo.
Parámetro	Nombre declarado para recibir un argumento.
Proceso	Instancia de ejecución con memoria y recursos.
Puntero	Valor de dirección utilizado para acceder a un objeto válido.
Retorno	Resultado que una función entrega al llamador.
Servicio	Capacidad ofrecida a usuarios u otras aplicaciones.
Traza	Seguimiento de valores y decisiones paso a paso.
Validación	Comprobación de formato, rango y reglas.

GiB significa 1024 × 1024 × 1024 bytes. Un tamaño lógico de archivo no equivale a la ocupación física en bloques. Una consulta de directorio tampoco garantiza que el contenido de cada archivo pueda leerse.

40 Comparación opcional con Python

Consulta de equivalencias sin cambiar el lenguaje principal del curso.

Idea	C	Python
Variable entera	int cantidad = 4;	cantidad = 4
Mostrar	printf("%d\n", cantidad);	print(cantidad)
Igualdad	a == b	a == b
Y lógico	a && b	a and b
División real	9.0 / 2	9 / 2
Repetir cinco veces	for (int i=0; i<5; i++)	for i in range(5):
Definir suma	int sumar(int a, int b)	def sumar(a, b):
Devolver	return a + b;	return a + b
Agrupar bloque	Llaves { }	Sangría
Texto	Array de char terminado en \0	Objeto str

Esta tabla no sustituye la explicación de cada lenguaje. Por ejemplo, los enteros de Python no tienen el mismo modelo de rango que int de C y las cadenas no se gestionan igual. La equivalencia describe la intención inicial de ejemplos pequeños, no todos sus detalles.

Cómo aprovechar el manual anterior

Si encuentras un ejemplo antiguo en Python, identifica primero el algoritmo, las entradas y los errores que contempla. Después utiliza la versión C de este manual. No traduzcas palabra por palabra imports, excepciones, listas o aperturas de archivos sin elegir las estructuras y las interfaces apropiadas.

Python queda como ampliación opcional. Las prácticas principales y el proyecto se programan en C. No es necesario estudiar esta comparación para continuar con funciones, archivos y recursos del sistema.

Para practicar Explica con palabras el algoritmo de sumar dos valores. Señala qué partes pertenecen al razonamiento y cuáles cambian por la sintaxis del lenguaje.

Referencias y correspondencia de contenidos

Fuentes técnicas de consulta y relación didáctica con UF1286

Eje del manual anterior	Desarrollo en esta edición
Sistema operativo y servicios	Capítulos 2 a 4 y 19, 23 a 27.
Lenguajes bibliotecas e interfaces	5 a 8, 13 a 26 y comparación 40.
Requisitos y ciclo de vida	1, 28, 29 y 35.
Desarrollo calidad y optimización	9 a 22 y 29 a 38.
Puente hacia otras unidades	36 y prácticas de ampliación.

Se mantienen los ejes conceptuales del material anterior y se reescribe la implementación en C. El proyecto se simplifica de manera explícita: no ordena filas y no utiliza un analizador JSON. Las interfaces Linux POSIX se diferencian de C estándar. El docente completa y evalúa los contenidos conforme a la programación y a la ficha del certificado.

Documentos técnicos

WG14 · Borrador público N1570 de C11. Referencia para funciones, tipos y biblioteca estándar. Apartados útiles: 5.1.2.2.1 inicio de programa, 6.5.2.2 llamadas, 7.21 entrada y salida y 7.22 utilidades. https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf

GNU GCC · Opciones de diagnóstico y advertencias. https://gcc.gnu.org/onlinedocs/gcc/Warning-Options.html

The Open Group · Especificaciones POSIX. Referencia de opendir, readdir, fstatat, statvfs, fork, exec y waitpid. https://pubs.opengroup.org/onlinepubs/9799919799/

Referencia del certificado · Real Decreto 686/2011, anexo VI y modificaciones aplicables. La correspondencia del manual es orientativa. https://www.boe.es/diario_boe/txt.php?id=BOE-A-2011-10055

Edición revisada el 10 de noviembre de 2026. Ejemplos de elaboración didáctica original. Las interfaces y sus límites deben comprobarse en la documentación del entorno del aula.